Teoría de Algoritmos
(TB024) Curso Echevarría

Trabajo Práctico 1

26 de abril de 2026

Integrantes	Padrón
Joaquín Hernández	110470
Juan Francisco Cuevas	107963
Alexander Terrussi	107212
Dante Finci	00
Víctor Zacarías	107080

Índice

1. Introducción General **4**

2. Ejercicio #1: Comparación de Monedas **5**
2.1. Problema a resolver 5
2.2. Algoritmo de División y Conquista 5
2.3. Complejidad temporal y cantidad de pesadas 5
2.4. Metodología adoptada 6
2.5. Resultados y gráficos 6
2.6. Relación entre tiempo de ejecución y pesadas 6

3. Ejercicio #2: Algoritmo Paolo **7**
3.1. Algoritmo 7
3.2. Estructuras de datos 9
3.3. Naturaleza Greedy del algoritmo 9
3.4. Complejidad temporal y espacial 9

4. Ejercicio #3: Algoritmo Aspiradora Robot **10**
4.1. Introducción y Problema 10
4.2. Algoritmo de Backtracking 10
4.3. Supuestos 10
4.4. Diseño 10
4.5. Seguimiento 12
4.6. Resultados experimentales 12
4.7. Complejidad temporal y espacial 13
4.8. Relación con los resultados 13

5. Ejercicio #4: Búsqueda de palíndromos **14**
5.1. Supuestos tomados para la resolución del algoritmo 14
5.2. Diseño del Algoritmo 14
5.2.1. Definición del Subproblema 14
5.2.2. Ecuación de Recurrencia 14
5.2.3. Preprocesamiento de Palíndromos 14
5.2.4. Subestructura Óptima 15
5.2.5. Subproblemas Superpuestos 15
5.2.6. Memorización 15
5.2.7. Código en Python 15
5.3. Seguimiento de ejemplo 15
5.4. Complejidad 16
5.5. Sets de Datos 16
5.6. Tiempos de Ejecución 16
5.7. Gráfico de los tiempos de ejecución 16

5.8. Resultados Experimentales 16
5.9. Análisis de los Resultados 17
5.10. Comparación con la Curva Teórica 17
5.11. Conclusión Experimental 17
5.12. Conclusión 18

1. Introducción General

Este informe detalla la resolución de cuatro problemas algorítmicos utilizando diferentes técnicas: División y Conquista, Algoritmos Greedy, Backtracking y Programación Dinámica. Cada sección analiza el problema propuesto, describe el algoritmo diseñado y evalúa su complejidad teórica.

2. Ejercicio #1: Comparación de Monedas

2.1. Problema a resolver

Se tiene una bolsa con n monedas de igual denominación, de las cuales exactamente una es falsa y es más liviana que el resto. El objetivo es identificar la moneda falsa sin ordenar por peso y usando División y Conquista.

2.2. Algoritmo de División y Conquista

La entrada del algoritmo es una lista de monedas representadas por su peso. Se asume que todos los pesos son iguales, a excepción del peso de la moneda falsa.

El algoritmo divide la lista en dos. En cada paso compara las monedas de las posiciones centrales:

1. Si las dos monedas centrales son distintas, la más liviana es la falsa.
2. Si son iguales, se llama recursivamente a las dos partes y se repite el algoritmo.

La cantidad de pesadas se contabiliza con cada comparación entre dos monedas, representando la cantidad de veces que se pesan dos monedas hasta conseguir la falsa.

El código del algoritmo implementado para resolver el problema es:

```python
def encontrar_moneda_falsa_dyc(monedas, inicio, fin, contador):
    if inicio >= fin:
        return monedas[inicio]

    if fin - inicio == 1:
        contador[0] += 1
        return monedas[inicio] if monedas[inicio] <= monedas[fin] else monedas[fin]

    medio = (inicio + fin) // 2

    contador[0] += 1
    if monedas[medio] != monedas[medio + 1]:
        return monedas[medio] if monedas[medio] < monedas[medio + 1] else monedas[
    medio + 1]

    izquierdo = encontrar_moneda_falsa_dyc(monedas, inicio, medio, contador)
    derecho = encontrar_moneda_falsa_dyc(monedas, medio + 1, fin, contador)

    contador[0] += 1
    return izquierdo if izquierdo <= derecho else derecho
```

2.3. Complejidad temporal y cantidad de pesadas

Dado que el algoritmo se trata de un algoritmo de Divide y Conquista, la complejidad viene determinada por la ecuación de recurrencia del mismo:

$T(n) = A.T(n/B) + O(n^C)$

Donde, $A = 2$ debido a que se hacen dos llamadas recursivas en cada paso, $B = 2$ porque en cada paso el algoritmo parte el problema en dos, y $C = 0$ porque los costos adicionales incurridos en cada paso son $O(1)$.

La ecuación de recurrencia resultante es:

$T(n) = 2T(n/2) + O(1)$

Por el teorema maestro, la complejidad del algoritmo es: $O(n)$

2.4. Metodología adoptada

Se evaluó el algoritmo con sets de datos de distintos tamaños $n = \{100, 500, 1000, 5000, 10000, 50000\}$. Para cada tamaño se generó un archivo con una única moneda falsa y se ejecutó:

```
python3 ejercicios/ej-dyc/monedas.py <tamaño_set_de_datos>
```

El tiempo se midió con la función time del sistema y se registró la cantidad de pesadas. Los resultados se guardaron en una tabla y se graficaron junto con una curva teórica lineal $O(n)$.

2.5. Resultados y gráficos

n	Tiempo (s)	Pesadas
100	0.000021	160
500	0.000080	746
1000	0.000162	1507
5000	0.000904	7945
10000	0.001782	15900
50000	0.010770	82764

Cuadro 1: Resultados experimentales del ejercicio 1.

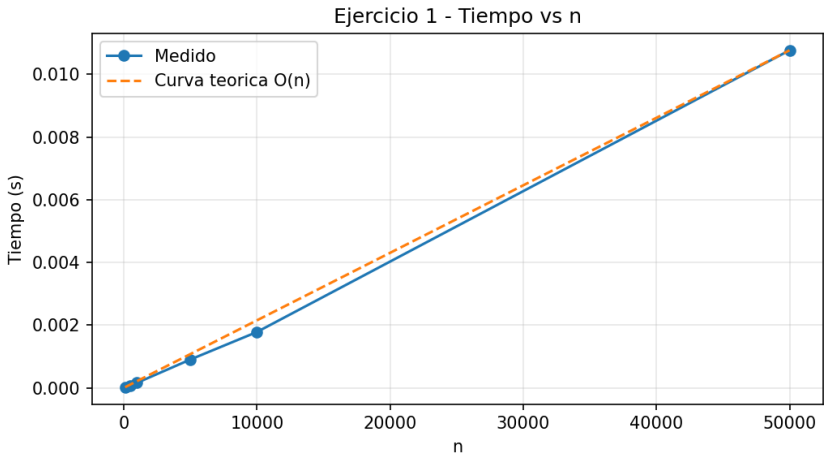


Figura 1: Tiempo de ejecución medido vs curva teórica $O(n)$.

Los gráficos muestran un crecimiento cercano al lineal, consistente con la complejidad analizada. La curva teórica se escaló para superponerla con la curva empírica obtenida a partir de la ejecución del algoritmo con distintos sets de datos.

2.6. Relación entre tiempo de ejecución y pesadas

Es razonable vincular el tiempo de ejecución del algoritmo con la cantidad de pesadas debido a que cada paso del algoritmo implica realizar una cantidad constante de pesadas, y el algoritmo tiene un crecimiento lineal, por lo tanto la cantidad de pesadas crece de forma proporcional al algoritmo, es decir, linealmente ($O(n)$).

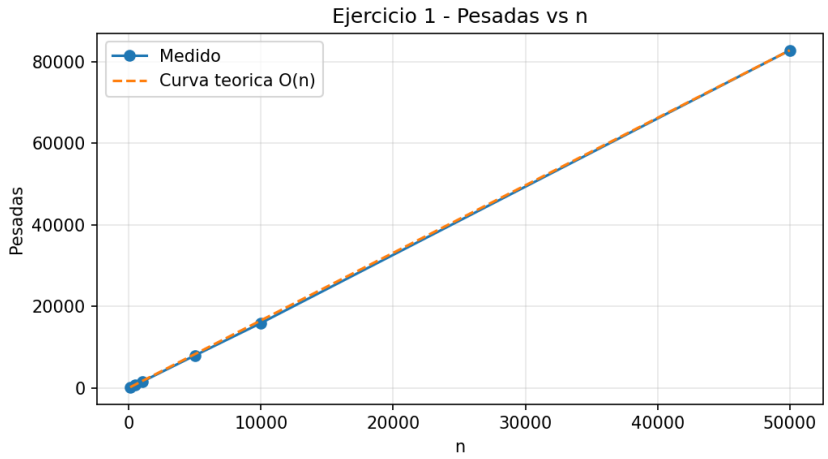


Figura 2: Cantidad de pesadas medida vs curva teórica $O(n)$.

3. Ejercicio #2: Algoritmo Paolo

Se tiene un algoritmo escrito en un lenguaje inusual, el objetivo es el de identificar el problema que resuelve, analizando su comportamiento greedy, el uso de estructuras de datos y la complejidad del mismo.

Adicionalmente, el algoritmo será traducido a un lenguaje de programación más moderno, Python.

3.1. Algoritmo

El problema resuelto por el algoritmo es el de caminos mínimos desde un único origen en un grafo no dirigido con pesos no negativos. El mismo fue implementado con el siguiente código:

```
10 CLS
20 INPUT "INGRESE EL NUMERO DE VERTICES "; N
30 PRINT
40 DIM COST (N,N)
50 DIM DIST (N)
60 DIM SOL (N)
70 A=1
80 PRINT "INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)"
90 PRINT
100 INPUT "EL EJE "; A, B
110 IF A=0 THEN 140
120 INPUT "COSTO DEL EJE "; COST(A,B)
130 GOTO 100
140 FOR I=1 TO N
150 FOR J=1 TO N
160 IF COST (I,J)=0 THEN COST (I,J)=15000
170 NEXT J
180 NEXT I
190 FOR I=1 TO N
200 FOR J=1 TO I
210 COST (I,J)=COST (J,I)
220 NEXT J
230 NEXT I
240 INPUT "INGRESE EL VERTICE DE SALIDA "; V
250 FOR I=1 TO N
260 DIST(I)=COST(V,I)
270 SOL(I)=0
280 NEXT I
290 GOSUB 1000
300 PRINT "SALIDA","LLEGADA","DISTANCIA"
```

```
310 FOR I=1 TO N
320 IF DIST(I)<15000 THEN PRINT V, I, DIST(I)
330 NEXT I
340 INPUT "OTRA VEZ? (SI/NO) "; RES$
350 IF RES$="NO" THEN END
360 FOR I=1 TO N
370 SOL(I)=0: DIST(I)=0
380 NEXT I
390 GOTO 240
1000 SOL(V)=1
1010 DIST(V)=0
1020 FOR I=1 TO N-1
1030 U=15000
1040 FOR J=1 TO N
1050 IF DIST(J)<=U AND SOL(J)=0 THEN U=J
1060 NEXT J
1070 SOL(U)=1
1080 FOR J=1 TO N
1090 IF DIST(J)>= (DIST(U)+COST(U,J)) THEN DIST(J)=DIST(U)+COST(U,J)
1100 NEXT J
1110 NEXT I
1120 RETURN
```

Listing 1: Algoritmo Paolo original

Su traducción más literal a un lenguaje moderno (Python) es:

```

def paolo():
    n = int(input("INGRESE EL NUMERO DE VERTICES "))
    cost = [[0 for _ in range(n)] for _ in range(n)]
    dist = [0 for _ in range(n)]
    sol = [0 for _ in range(n)]
    a = 1
    print("INGRESE EL CUADRO DE COSTOS (INGRESE 0,0 PARA TERMINAR)")
    print()
    while True:
        a, b = map(int, input("EL EJE ").split())
        if a == 0 and b == 0:
            break
        cost[a][b] = int(input("COSTO DEL EJE "))
    for i in range(n):
        for j in range(n):
            if cost[i][j] == 0:
                cost[i][j] = 15000
    for i in range(n):
        for j in range(i):
            cost[i][j] = cost[j][i]
    v = int(input("INGRESE EL VERTICE DE SALIDA "))
    for i in range(n):
        dist[i] = cost[v][i]
        sol[i] = 0
    paolo_sub(n, cost, dist, sol, v)
    print("SALIDA", "LLEGADA", "DISTANCIA")
    for i in range(n):
        if dist[i] < 15000:
            print(v, i, dist[i])
    res = input("OTRA VEZ? (SI/NO) ")
    if res == "NO":
        return
    for i in range(n):
        sol[i] = 0
        dist[i] = 0
    paolo()

def paolo_sub(n, cost, dist, sol, v):
    sol[v] = 1
    dist[v] = 0
    for i in range(n - 1):
        u = 15000
        for j in range(n):
```

```
            if dist[j] <= u and sol[j] == 0:
                u = j
        sol[u] = 1
        for j in range(n):
            if dist[j] >= (dist[u] + cost[u][j]):
                dist[j] = dist[u] + cost[u][j]

if __name__ == "__main__":
    paolo()
```

Listing 2: Algoritmo Paolo traducido

El algoritmo crea una matriz de adyacencia para representar al grafo, donde cada entrada $cost[i][j]$ almacena el costo de viajar desde el vértice i al vértice j. Si un costo no se ingresa, se asume un valor predeterminado de 15000 (considerado infinito en este contexto). Luego, la matriz se ajusta para que sea simétrica, lo que indica que el grafo es no dirigido.

A partir de un vértice de salida v, calcula las distancias más cortas a todos los demás vértices utilizando una variante del algoritmo de Dijkstra. En cada iteración, selecciona el vértice no visitado con menor distancia y actualiza las distancias del resto usando ese vértice como punto de partida.

3.2. Estructuras de datos

Se utilizan una matriz de adyacencia y dos arreglos. La matriz *cost* permite consultar en tiempo constante el peso de cualquier arista, el arreglo *dist* guarda las mejores distancias encontradas hasta el momento, y el arreglo *sol* indica qué vértices ya fueron incorporados definitivamente a la solución.

3.3. Naturaleza Greedy del algoritmo

En cada paso del algoritmo se toma la misma decisión, se elige el vértice no visitado con menor distancia desde el origen, esta regla constituye una regla Greedy ya que se está tomando una decisión en base a un óptimo local (çuál es la menor distancia a la que me puedo mover"), la cual en conjunto al resto de pasos del algoritmo que replican la misma regla buscan acercarse al óptimo global.

3.4. Complejidad temporal y espacial

La complejidad temporal es $O(n^2)$ y la complejidad espacial es $O(n^2)$. El uso de memoria está dominado por la matriz de adyacencia de tamaño $n \times n$. En tiempo, el algoritmo realiza para cada uno de los n vértices una búsqueda lineal del siguiente vértice a fijar y una recorrida lineal para mejorar las distancias, resultando en un costo cuadrático.

4. Ejercicio #3: Algoritmo Aspiradora Robot

4.1. Introducción y Problema

En este ejercicio se modela el recorrido de una aspiradora robot dentro de un laberinto rectangular. El laberinto se representa mediante una matriz, donde cada celda puede corresponder a una pared (X), una posición transitable, la entrada (E) o la salida (S).

El objetivo es diseñar, implementar y documentar un algoritmo de backtracking que, dado un laberinto de tamaño $m \times n$, encuentre un camino válido desde la posición inicial E hasta la salida S, en caso de que dicho camino exista.

La solución devuelve el recorrido como una secuencia de coordenadas de la matriz. A partir de ese camino, pueden interpretarse los movimientos que debería realizar la aspiradora para desplazarse por el laberinto, utilizando sus operaciones disponibles: observar la celda siguiente, avanzar y girar.

4.2. Algoritmo de Backtracking

El problema se resuelve mediante backtracking. Desde la entrada E, el algoritmo intenta avanzar recursivamente hacia las posiciones vecinas en un orden fijo: derecha, izquierda, abajo y arriba.

Antes de avanzar, se verifica que la celda esté dentro de los límites, que no sea una pared X y que no haya sido visitada previamente. Si la celda es válida, se agrega a la solución y se marca como visitada.

Cuando se alcanza la salida S, se devuelve el camino encontrado. Si desde una posición no se puede continuar hacia la salida, se elimina esa posición de la solución y se vuelve atrás para probar otra alternativa.

4.3. Supuestos

Se asume que el laberinto es rectangular, por lo que todas sus filas tienen la misma cantidad de columnas. Sus dimensiones se obtienen a partir del archivo de entrada.

También se asume que existe una única posición inicial E y al menos una salida S. Las celdas marcadas con X representan paredes y no pueden ser recorridas. Del mismo modo, cualquier posición fuera de los límites del laberinto se considera inválida.

En caso de que no exista un camino posible entre la entrada y la salida, el algoritmo devuelve una lista vacía.

4.4. Diseño

La solución se basa en un algoritmo recursivo de backtracking. En cada llamada se analiza una posición del laberinto y se verifica si puede ser recorrida. Una posición es válida si está dentro de los límites de la matriz, no corresponde a una pared y no fue visitada anteriormente.

Si la posición es válida, se agrega a la solución y se marca como visitada. Luego se prueban las cuatro direcciones posibles en un orden fijo: derecha, izquierda, abajo y arriba. Si alguna de esas ramas alcanza la salida, el algoritmo termina y devuelve el camino encontrado. En caso contrario, se elimina la posición actual de la solución y se retrocede para probar otras alternativas.

Las estructuras de datos utilizadas son:

- Una matriz de caracteres para representar el laberinto.
- Una lista con la solución construida hasta el momento.
- Un conjunto de posiciones visitadas para evitar ciclos y no recorrer dos veces la misma celda.

Código en Python:

```python
DIRECCIONES = (
    (0, 1),
    (0, -1),
    (1, 0),
    (-1, 0),
)


def obtener_inicio(laberinto):
    if not laberinto:
        return None, None

    for fila in range(len(laberinto)):
        for columna in range(len(laberinto[0])):
            if laberinto[fila][columna] == "E":
                return fila, columna

    return None, None


def posicion_valida(laberinto, fila, columna, visitados):
    return (
        fila >= 0
        and columna >= 0
        and fila < len(laberinto)
        and columna < len(laberinto[0])
        and laberinto[fila][columna] != "X"
        and (fila, columna) not in visitados
    )


def avanzar(laberinto, fila, columna, resultado, visitados):
    if not posicion_valida(laberinto, fila, columna, visitados):
        return False

    resultado.append((fila, columna))
    visitados.add((fila, columna))

    if laberinto[fila][columna] == "S":
        return True

    for desplazamiento_fila, desplazamiento_columna in DIRECCIONES:
        nueva_fila = fila + desplazamiento_fila
        nueva_columna = columna + desplazamiento_columna

        if avanzar(laberinto, nueva_fila, nueva_columna, resultado, visitados):
            return True

    resultado.pop()
    return False


def encontrar_camino_bt(laberinto):
    fila_inicial, columna_inicial = obtener_inicio(laberinto)

    if fila_inicial is None or columna_inicial is None:
        return []

    resultado = []
    visitados = set()

    encontro_salida = avanzar(
        laberinto,
        fila_inicial,
        columna_inicial,
        resultado,
        visitados,
    )

```

```
    if encontro_salida:
        return resultado

    return []
```

El código del algoritmo se encuentra en *code/laberinto.py*. La función *avanzar* concentra la lógica recursiva del backtracking, mientras que *encontrar_camino_bt* busca la posición inicial y actúa como punto de entrada del algoritmo.

4.5. Seguimiento

Para el seguimiento se toma el archivo *laberinto1.txt*.

Las posiciones se indican con índices base cero. Por lo tanto, la entrada E se encuentra en la posición $(5, 1)$ y la salida S en la posición $(1, 7)$.

En el laberinto analizado, la entrada E se encuentra en la posición $(5, 1)$ y la salida S en la posición $(1, 7)$.

A partir de ahí, continúa explorando recursivamente las posiciones válidas. Cada celda recorrida se agrega a la solución y se marca como visitada. Cuando una rama no permite llegar a la salida, el algoritmo retrocede y prueba otra dirección.

El camino devuelto para este caso es:

```
[(5, 1), (4, 1), (3, 1), (3, 2), (3, 3), (4, 3), (4, 4),
 (4, 5), (4, 6), (3, 6), (2, 6), (1, 6), (1, 7)]
```

Este resultado muestra que el algoritmo encuentra un camino válido desde la entrada hasta la salida, sin atravesar paredes ni repetir celdas.

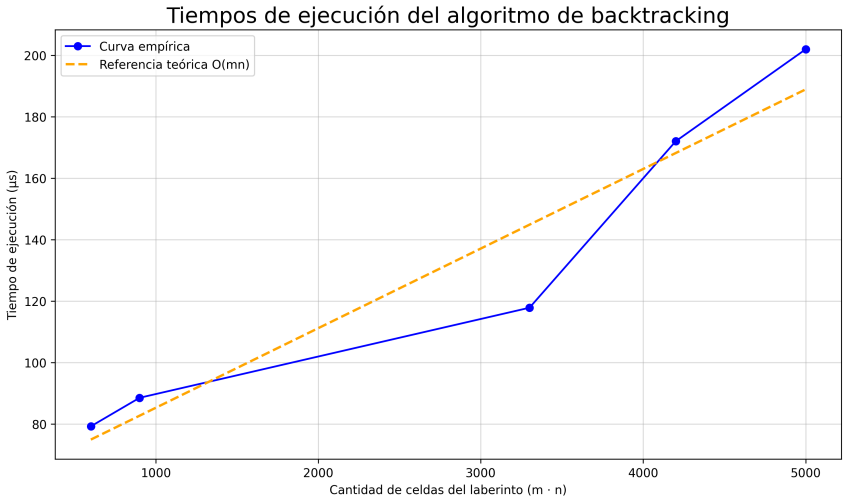
4.6. Resultados experimentales

Para validar el comportamiento del algoritmo, se ejecutó la solución sobre cinco conjuntos de datos con laberintos de distinto tamaño. En todos los casos, el algoritmo finalizó correctamente y devolvió un camino válido desde la entrada E hasta la salida S. Para las mediciones se utilizó el módulo *ejercicios/ej-backtracking/laberinto_informe.py*, que imprime el tiempo de ejecución.

Los tiempos de ejecución obtenidos fueron los siguientes:

Caso	Tiempo de ejecución
Laberinto 1	79,209 μs
Laberinto 2	88,500 μs
Laberinto 3	117,834 μs
Laberinto 4	172,042 μs
Laberinto 5	202,000 μs

A continuación se presenta un gráfico comparativo de los tiempos medidos para cada conjunto de datos.



Como se observa en la tabla y en el gráfico, los tiempos de ejecución presentan una tendencia creciente a medida que aumenta el tamaño de los laberintos. Esto resulta coherente con el análisis teórico, ya que para entradas más grandes el algoritmo puede requerir explorar una mayor cantidad de posiciones antes de encontrar la salida.

4.7. Complejidad temporal y espacial

Sea m la cantidad de filas del laberinto y n la cantidad de columnas. En el peor caso, el algoritmo puede llegar a analizar todas las celdas de la matriz.

Como las posiciones visitadas se almacenan en un conjunto, cada celda se analiza a lo sumo una vez y la consulta para saber si ya fue visitada tiene costo promedio $O(1)$. Para cada posición válida se prueban hasta cuatro direcciones posibles: derecha, izquierda, abajo y arriba. Como la cantidad de direcciones es constante, la complejidad temporal resulta:

$$O(mn)$$

La complejidad espacial también es $O(mn)$. Esto se debe a que el algoritmo almacena la solución y el conjunto de posiciones visitadas. En el peor caso, ambas estructuras pueden crecer proporcionalmente a la cantidad total de celdas del laberinto.

Por lo tanto, tanto la complejidad temporal como la espacial del algoritmo son $O(mn)$.

4.8. Relación con los resultados

Los tiempos medidos son bajos, por lo que pueden verse afectados en parte por factores propios de la ejecución y de la medición. Sin embargo, al considerar los cinco conjuntos de datos, se observa una tendencia creciente a medida que aumenta el tamaño del laberinto.

Estos resultados son compatibles con la complejidad temporal, $O(mn)$, ya que el algoritmo recorre como máximo una vez cada celda del laberinto y, para cada una, prueba una cantidad constante de direcciones.

De todos modos, el tiempo de ejecución no depende únicamente de la cantidad total de celdas, sino también de la distribución de paredes y de la ubicación de la salida. Si la salida se encuentra rápidamente, el algoritmo puede finalizar sin recorrer toda la matriz. En cambio, si el camino requiere explorar más alternativas, el tiempo aumenta.

Por lo tanto, los resultados experimentales acompañan el análisis teórico: en los casos evaluados, al aumentar el tamaño del laberinto también aumenta el tiempo de ejecución, manteniéndose dentro del comportamiento esperado para una búsqueda con backtracking sobre una matriz.

5. Ejercicio #4: Búsqueda de palíndromos

Para este ejercicio se busca resolver mediante programación dinámica el problema de obtener la menor cantidad de palíndromos dentro de un string.

5.1. Supuestos tomados para la resolución del algoritmo

- El algoritmo tiene como entrada un string longitud $n \geq 1$.
- Un string de longitud 1 es un palíndromo, por ejemplo, A es un palíndromo.
- El acceso a caracteres es $O(1)$ dentro del string.

5.2. Diseño del Algoritmo

Para la resolución del algoritmo mediante programación dinámica fue necesario analizar en primer lugar la forma que tienen los subproblemas del problema a resolver, y en segundo la ecuación de recurrencia del mismo.

5.2.1. Definición del Subproblema

Sea:

$$dp[i] = \text{mínima cantidad de palíndromos necesarios para } S[0..i],$$

$S[0..i]$: El string desde la posición 0 hasta la posición i.

5.2.2. Ecuación de Recurrencia

$$dp[i] = \min_{0 \leq j \leq i} \{dp[j-1] + 1 \mid \text{Si S[j..i] es palíndromo}\}$$

Condición base:

$$dp[-1] = 0$$

Nota: Para una string que va desde 0 hasta i la ecuación de recurrencia busca obtener la mínima cantidad de palíndromos que hay en todos los substrings que hay desde 0 hasta i.

5.2.3. Preprocesamiento de Palíndromos

Como paso inicial, se define una matriz:

$$pal[i][j] = \begin{cases} True & \text{si } S[i..j] \text{ es palíndromo} \\ False & \text{en caso contrario} \end{cases}$$

$$pal[i][j] = (S[i] = S[j]) \wedge (j - i \leq 1 \vee pal[i+1][j-1])$$

Esta permite evaluar si el string que va desde i hasta j es un palíndromo.

La matriz sirve como estructura de datos auxiliar para el cálculo de la cantidad mínima de palíndromos para cada tamaño de subproblemas, evitando realizar cálculos de forma repetitiva.

5.2.4. Subestructura Óptima

La solución óptima para $S[0..i]$ depende de soluciones óptimas de subproblemas $S[0..j-1]$. Esto cumple con la propiedad de la programación dinámica de resolver un problema a partir del espacio de subproblemas del mismo.

5.2.5. Subproblemas Superpuestos

Los valores de $dp[i]$ y $pal[i][j]$ se reutilizan múltiples veces, evitando recomputaciones.

5.2.6. Memorización

Se utilizan:

- Matriz $pal[n][n]$
- Lista $dp[n]$

La matriz, tal como se comentó en la sección "Preprocesamiento de Palíndromos"se usa para almacenar que substrings del string original son palíndromos. La lista de óptimos almacena la mínima cantidad de palíndromos para todos los substrings de la forma $[0...i]$.

La principal diferencia con la matriz, es que la matriz guarda todas las combinaciones de substring, y la ecuación de recurrencia solo los que van de 0 hasta i (subproblemas del algoritmo).

5.2.7. Código en Python

```python
def min_palindromos(S):
    n = len(S)
    if n == 0:
        return 0

    pal = [[False] * n for _ in range(n)]

    for i in range(n):
        pal[i][i] = True

    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length - 1

            if S[i] == S[j] and (length == 2 or pal[i + 1][j - 1]):
                pal[i][j] = True

    dp = [0] * n

    for i in range(n):
        if pal[0][i]:
            dp[i] = 1
        else:
            dp[i] = float("inf")
            for j in range(1, i + 1):
                if pal[j][i]:
                    dp[i] = min(dp[i], dp[j - 1] + 1)

    return dp[n - 1]
```

5.3. Seguimiento de ejemplo

Ejemplo: $S = ABA$

- $dp[0] = 1$

- $dp[1] = 2$
- $dp[2] = 1$

Resultado: 1 partición.

5.4. Complejidad

Para analizar la complejidad del algoritmo es necesario considerar la complejidad incurrida en cada parte del mismo. Se cuentan con dos partes principales, el armado de la matriz y el cálculo de los óptimos a partir de la ecuación de recurrencia.

- Cálculo de palíndromos: $O(n^2)$
- Programación dinámica: $O(n^2)$

Complejidad total:

$$O(n^2)$$

5.5. Sets de Datos

Se utilizaron los siguientes tipos de datos para estudiar al algoritmo:

- Mejor caso: cadenas repetidas (ej: AAAA)
- Peor caso: caracteres distintos (ej: ABCDEF)
- Casos aleatorios con semilla fija

5.6. Tiempos de Ejecución

Se midieron tiempos utilizando múltiples ejecuciones por cada tamaño de entrada, promediando los resultados para cada tamaño de entrada.

5.7. Gráfico de los tiempos de ejecución

Se obtuvo un crecimiento cuadrático en los tiempos de ejecución, consistente con la complejidad teórica.

5.8. Resultados Experimentales

A continuación se presentan los resultados obtenidos a partir de la ejecución del algoritmo sobre distintos tamaños de entrada:

n	Resultado	Tiempo Promedio (s)
10	5	0.00000941
50	13	0.00013710
100	26	0.00053040
200	61	0.00291414
400	127	0.00985342
800	261	0.03777179

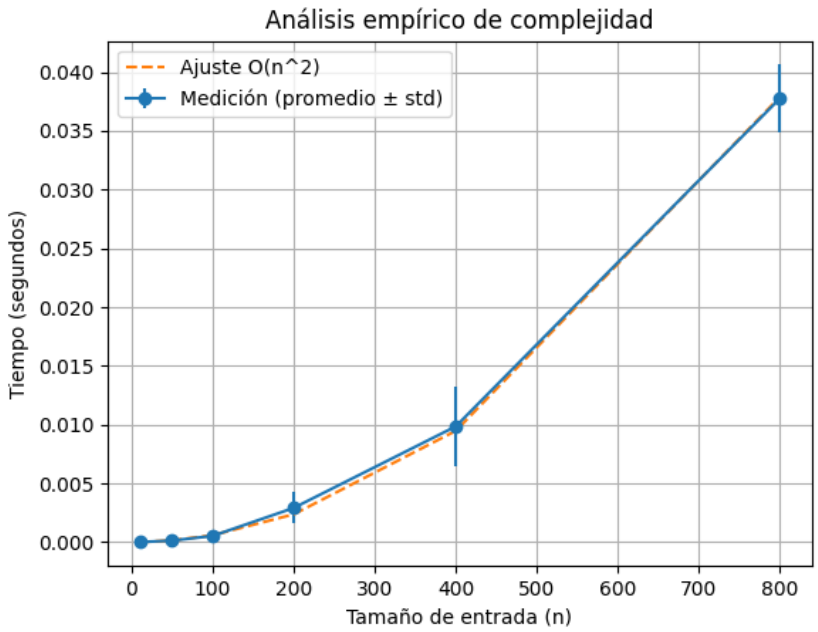


Figura 3: Comparación entre tiempos medidos y curva teórica

5.9. Análisis de los Resultados

A partir de los datos obtenidos, se observa que el tiempo de ejecución crece de manera consistente al aumentar el tamaño de la entrada. En particular, el crecimiento no es lineal, sino que se acelera a medida que n aumenta, lo cual es indicativo de un comportamiento cuadrático.

Este resultado es coherente con el análisis teórico del algoritmo, cuya complejidad temporal es $O(n^2)$. Si se comparan los tiempos entre distintas escalas, se puede notar que al duplicar el tamaño de la entrada, el tiempo de ejecución tiende a multiplicarse aproximadamente por un factor cercano a 4, lo cual coincide con el comportamiento esperado de una función cuadrática.

Por ejemplo, al pasar de $n = 200$ a $n = 400$, el tiempo crece de aproximadamente 0,0029 segundos a 0,0098 segundos, lo cual representa un incremento cercano a cuatro veces. De manera similar, al duplicar nuevamente hasta $n = 800$, el tiempo alcanza aproximadamente 0,0377 segundos, manteniendo la misma tendencia.

5.10. Comparación con la Curva Teórica

Al comparar los tiempos medidos con la curva teórica de complejidad $O(n^2)$, se observa que ambos presentan una tendencia similar. Esto valida empíricamente el análisis teórico realizado previamente.

Si bien pueden existir pequeñas desviaciones, especialmente en tamaños pequeños donde los costos constantes tienen mayor impacto, en general la curva experimental se ajusta adecuadamente al modelo teórico.

5.11. Conclusión Experimental

Los resultados obtenidos confirman que el algoritmo presenta un comportamiento cuadrático en la práctica, tal como se había deducido teóricamente. Esto demuestra la efectividad del enfoque de programación dinámica utilizado, tanto desde el punto de vista conceptual como en términos de rendimiento.

5.12. Conclusión

A lo largo de este trabajo se pudo ver cómo un problema que en principio parece complejo, como probar todas las formas posibles de dividir una cadena, puede resolverse de manera eficiente utilizando programación dinámica.

La clave estuvo en cambiar la forma de pensar el problema: en lugar de evaluar todas las particiones posibles, se aprovechó la estructura del problema para construir la solución a partir de subproblemas más pequeños. Además, el preprocesamiento de palíndromos permitió evitar cálculos repetidos y mejorar significativamente el rendimiento.

En la práctica, los resultados obtenidos muestran que el tiempo de ejecución crece de manera cuadrática con el tamaño de la entrada, lo cual coincide con el análisis teórico realizado. Si bien para tamaños pequeños la diferencia no es tan evidente, a medida que crece el tamaño de la cadena se observa claramente esta tendencia.

En conclusión, el uso de programación dinámica permitió transformar un problema potencialmente exponencial en uno manejable, demostrando la importancia de identificar subestructuras óptimas y reutilizar resultados ya calculados.